

PYTHON

Industry + Interview Cheat Sheet

From Python Basics to OOP, Advanced Concepts, Files and Threading

★★★ MUST KNOW - High-frequency interview topic and day-to-day industry skill

★★ IMPORTANT - Frequently useful; understand well

★ USEFUL - Know the idea and syntax

Best use: revise the ★★★ sections first, then practice the examples without looking.

0. What to Prioritize for Interviews and Industry

Top interview priority: Functions and arguments; strings; lists/dictionaries/sets; comprehensions; OOP; exceptions; decorators; *args/**kwargs; scope; file handling; class/instance/static methods; inheritance; dunder methods; threading basics.

Top industry priority: Readable functions, data structures, error handling, context managers, modules/packages, HTTP requests, logging-ready file handling, CLI arguments, OOP where appropriate, and understanding concurrency trade-offs.

Rapid priority table

Tier	Topics	Why it matters
Tier 1 - ★★★	Functions, collections, strings, OOP, exceptions, decorators, args/kwargs, files	Most interview questions and production code depend on these.
Tier 2 - ★★	Loops, conditionals, typecasting, lambda, recursion, modules, map/filter/reduce, argparse, requests	Common implementation and debugging topics.
Tier 3 - ★	Walrus, match-case, pass, some set/tuple methods, basic threading API	Useful differentiators; less frequently the main interview topic.

1. Python Syntax and Core Basics

Master these before moving to data structures and OOP.

Syntax and Indentation

Priority: ★★★ | MUST KNOW

Python uses indentation to define blocks instead of braces. Standard indentation is 4 spaces. Statements usually end at the line break.

Syntax

```
if condition:
    statement
else:
    statement
```

Example

```
age = 22
if age >= 18:
    print("Adult")
else:
    print("Minor")
```

Interview focus: Why indentation matters; Python is dynamically typed; blocks are created after a colon.

Industry use: Consistent formatting makes code review and maintenance much easier.

Common trap: Mixing tabs and spaces can raise `IndentationError` or create confusing layout.

Variables

Priority: ★★★ | MUST KNOW

A variable is a name bound to an object. Python does not require a type declaration; the type belongs to the object, not the variable name.

Syntax

```
name = value
x = 10
name = "Aman"
```

Example

```
x = 10
x = "ten"      # same name can point to a different type
a = b = 0
x, y = 5, 9
```

Interview focus: Dynamic typing; naming rules; multiple assignment; object references.

Industry use: Clear variable names reduce bugs. Prefer snake_case for variables and functions.

Common trap: Do not confuse assignment (=) with equality (==).

Type Casting

Priority: ★★★ | MUST KNOW

Type casting converts a value from one type to another using constructors such as `int()`, `float()`, `str()`, `list()`, `tuple()`, and `set()`.

Syntax

```
int(value)
float(value)
str(value)
bool(value)
```

Example

```
age_text = "25"
age = int(age_text)
price = float("99.50")
text = str(123)
```

```
print(bool(0), bool(1))    # False True
```

Interview focus: Explicit conversion vs automatic numeric promotion; ValueError when conversion is invalid.

Industry use: User input, APIs, CSV files, environment variables and command-line arguments often arrive as strings.

Common trap: `int("12.5")` fails. Use `int(float("12.5"))` only when that conversion is really intended.

User Input

Priority: ★★ | IMPORTANT

`input()` reads text from standard input and always returns a string. Convert it when a numeric or other type is required.

Syntax

```
value = input("Prompt: ")
```

Example

```
name = input("Name: ")
age = int(input("Age: "))
print(f"{name} will be {age + 1} next year")
```

Interview focus: What type does `input()` return? Always `str`.

Industry use: Useful for simple CLI programs and learning exercises.

Common trap: Never assume user input is valid; production code should validate and handle exceptions.

Comments

Priority: ★ | USEFUL

Comments explain intent. A single-line comment starts with `#`. Triple-quoted strings are strings, not true comments, though they are often used as documentation strings.

Syntax

```
# Single-line comment

"""Docstring or multi-line string"""
```

Example

```
# Calculate final amount after discount
final_amount = price * (1 - discount)
```

Interview focus: Difference between comments and docstrings.

Industry use: Good comments explain why, not obvious line-by-line behavior.

Common trap: Avoid stale comments that disagree with the code.

2. Operators

Arithmetic and Comparison Operators

Priority: ★★★ | MUST KNOW

Arithmetic operators perform numeric work; comparison operators return Boolean values.

Syntax

```
+ - * / // % **
== != > < >= <=
```

Example

```
a, b = 7, 2
print(a / b)    # 3.5
print(a // b)   # 3
print(a % b)    # 1
print(a ** b)   # 49
print(a >= b)   # True
```

Interview focus: Difference between `/` and `//`; `==` vs `is`; modulus use cases.

Industry use: Common in validation, pagination, batching, calculations and conditions.

Common trap: `is` checks identity, not value equality. Use `==` for normal value comparisons.

Logical Operators

Priority: ★★★ | MUST KNOW

and, or and not combine or invert Boolean expressions. Python short-circuits: it may skip evaluating later expressions once the result is known.

Syntax

```
condition1 and condition2
condition1 or condition2
not condition
```

Example

```
age = 24
has_id = True
if age >= 18 and has_id:
    print("Allowed")

name = ""
print(name or "Guest")  # Guest
```

Interview focus: Short-circuit behavior; truthy/falsy values; `and`/`or` can return operands, not only True/False.

Industry use: Used heavily in validation, fallback values, permissions and guards.

Common trap: Expressions such as `x or default` return one of the operands.

Assignment Operators

Priority: ★★ | IMPORTANT

Assignment operators update variables. Augmented assignment combines an operation with assignment.

Syntax

```
=  +=  -=  *=  /=  //=  %=  **=  &=  |=  ^=  >>=  <<=
```

Example

```
count = 10
count += 1
count *= 2
print(count)  # 22
```

Interview focus: Know += and other common augmented operators.

Industry use: Counters, accumulators, state updates.

Common trap: For mutable objects, += can behave differently from + by mutating in place in some cases.

Membership Operators

Priority: ★★★ | MUST KNOW

`in` checks whether a value exists in a collection; `not in` checks absence. With dictionaries, membership checks keys by default.

Syntax

```
value in collection
value not in collection
```

Example

```
roles = {"admin", "editor"}
print("admin" in roles)

user = {"name": "Riya", "age": 25}
print("name" in user)  # checks keys
```

Interview focus: Dictionary membership checks keys, not values.

Industry use: Very common for validation, allow-lists, deduplication and routing.

Common trap: For dictionary values use `value in d.values()`.

3. Conditionals and Pattern Matching

if / elif / else

Priority: ★★★ | MUST KNOW

Conditionals execute different blocks depending on Boolean conditions.

Syntax

```
if condition:
    ...
elif other_condition:
    ...
else:
    ...
```

Example

```
score = 78
if score >= 90:
    grade = "A"
elif score >= 75:
    grade = "B"
else:
    grade = "C"
```

Interview focus: Order matters; use chained comparisons such as `18 <= age < 60`.

Industry use: Business rules, validations, branching workflows.

Common trap: Too many nested if statements usually signal that code should be refactored into functions or mappings.

Conditional Expression (Ternary)

Priority: ★★ | IMPORTANT

A compact expression chooses one of two values based on a condition.

Syntax

```
value_if_true if condition else value_if_false
```

Example

```
age = 20
status = "Adult" if age >= 18 else "Minor"
```

Interview focus: Readable one-line conditions are useful, but nesting ternaries hurts readability.

Industry use: Small value selection and formatting logic.

match / case

Priority: ★★ | IMPORTANT

Structural pattern matching (Python 3.10+) compares a subject against patterns. It can replace some long if/elif chains.

Syntax

```
match value:
    case pattern1:
        ...
    case pattern2:
        ...
    case _:
        ...
```

Example

```
command = "start"
match command:
    case "start":
        print("Starting")
    case "stop":
```

```
print("Stopping")
case _:
    print("Unknown")
```

Interview focus: `\_` is the default wildcard. Pattern matching is more powerful than a classic switch.

Industry use: Useful in parsers, command handlers, structured data processing.

Common trap: Do not use match just to appear advanced; simple mappings or if/elif can be clearer.

4. Loops and Loop Control

for Loop

Priority: ★★★ | **MUST KNOW**

A for loop iterates over any iterable: list, tuple, string, dictionary, range, file object, generator and more.

Syntax

```
for item in iterable:
    ...
```

Example

```
names = ["Aman", "Riya", "Kabir"]
for index, name in enumerate(names, start=1):
    print(index, name)
```

Interview focus: Iterable vs iterator; range(); enumerate(); iterating dict keys/items.

Industry use: Processing collections, records, files, API responses.

Common trap: Do not modify a list structurally while iterating over it unless you intentionally iterate over a copy.

while Loop

Priority: ★★ | **IMPORTANT**

A while loop repeats as long as its condition remains truthy. It is useful when the number of iterations is not known in advance.

Syntax

```
while condition:
    ...
```

Example

```
attempts = 3
while attempts > 0:
    print("Try")
    attempts -= 1
```

Interview focus: Know how infinite loops happen and how to stop them.

Industry use: Polling, retries, state machines, interactive input loops.

Common trap: Always ensure the loop condition can eventually become false unless an intentional service loop is used.

break

Priority: ★★★ | **MUST KNOW**

break exits the nearest enclosing loop immediately.

Syntax

```
for item in items:
    if stop_condition:
        break
```

Example

```
for n in range(1, 10):
    if n == 5:
        break
    print(n)
```

Interview focus: break affects the nearest loop only.

Industry use: Search loops, retry logic, early termination.

continue

Priority: ★★ | IMPORTANT

continue skips the rest of the current iteration and moves to the next one.

Syntax

```
for item in items:
    if skip_condition:
        continue
    process(item)
```

Example

```
for n in range(6):
    if n % 2 == 0:
        continue
    print(n)    # 1 3 5
```

Interview focus: Difference between break and continue.

Industry use: Filtering invalid records during iteration.

pass

Priority: ★ | USEFUL

pass is a do-nothing placeholder. It allows syntactically valid empty blocks while code is being designed.

Syntax

```
if condition:
    pass
```

Example

```
class FutureFeature:
    pass

def todo():
    pass
```

Interview focus: pass does not skip an iteration; continue does.

Industry use: Temporary stubs and intentionally empty exception/class blocks.

Common trap: Avoid leaving pass in unfinished production logic without a clear reason.

Loop else

Priority: ★★ | IMPORTANT

A loop can have an else block. The else runs when the loop finishes normally, but not when it exits via break.

Syntax

```
for item in items:
    if found:
        break
else:
    ...
```

Example

```
target = 7
for n in [1, 3, 5]:
    if n == target:
        print("Found")
        break
else:
    print("Not found")
```

Interview focus: A classic interview question: when does loop-else execute?

Industry use: Search/validation logic where a normal completion has meaning.

5. Strings

String Basics and Immutability

Priority: ★★★ | **MUST KNOW**

A string is an immutable sequence of Unicode characters. Operations that appear to modify a string actually create a new string.

Syntax

```
text = "Python"
text = 'Python'
multiline = """line1
line2"""
```

Example

```
name = "Riya"
upper_name = name.upper()
print(name)      # Riya
print(upper_name) # RIYA
```

Interview focus: Strings are immutable and hashable; common methods return new strings.

Industry use: Text processing, APIs, logs, web data, file parsing.

Common trap: Repeated `result += piece` in large loops can be inefficient; use `"".join(parts)` for many pieces.

Indexing

Priority: ★★★ | **MUST KNOW**

Indexing retrieves one character. Positive indexes start at 0; negative indexes count from the end.

Syntax

```
text[index]
```

Example

```
text = "Python"
print(text[0])  # P
print(text[-1]) # n
```

Interview focus: IndexError occurs if the index is outside the string.

Industry use: Character-level parsing and validation.

Slicing

Priority: ★★★ | **MUST KNOW**

Slicing extracts a subsequence using start:stop:step. The stop index is excluded.

Syntax

```
text[start:stop:step]
```

Example

```
text = "Python"
print(text[1:4])  # yth
print(text[:3])   # Pyt
print(text[::2])  # Pto
print(text[::-1]) # nohtyP
```

Interview focus: Understand omitted start/stop, negative steps, and stop-exclusive behavior.

Industry use: Parsing prefixes/suffixes, chunks, reversed copies.

Common trap: Slicing silently clamps out-of-range bounds, while direct indexing can raise IndexError.

Common String Methods

Priority: ★★★ | MUST KNOW

String methods cover normalization, searching, replacement, splitting and validation.

Syntax

```
s.lower() s.upper() s.strip()
s.split(sep) sep.join(items)
s.replace(old, new)
s.startswith(x) s.endswith(x)
s.find(x) s.count(x)
s.isdigit() s.isalpha()
```

Example

```
raw = " Python,SQL,Git "
clean = raw.strip()
skills = clean.split(",")
print(skills)
print("-".join(skills))
print(clean.replace("Git", "Docker"))
```

Interview focus: Difference between find() and index(); split/join; strip does not remove an arbitrary substring.

Industry use: Normalizing input, parsing data, cleaning CSV/API fields.

Common trap: `strip("abc")` removes any of those characters from both ends; it does not remove the exact substring "abc".

f-Strings

Priority: ★★★ | MUST KNOW

Formatted string literals embed expressions directly inside strings. They are usually the cleanest formatting option.

Syntax

```
f"text {expression}"
```

Example

```
name = "Aman"
score = 92.456
print(f"{name}: {score:.2f}")
print(f"Next score: {score + 1:.1f}")
```

Interview focus: Formatting numbers, alignment, expressions, and why f-strings are preferred for readability.

Industry use: Logging messages, reports, CLI output and templates.

Common trap: For logging frameworks, parameterized logging is often better than eager f-string formatting when performance matters.

6. Functions, Arguments, Scope and Modules

One of the highest-value sections for interviews.

Functions

Priority: ★★★ | MUST KNOW

A function groups reusable behavior. It can accept parameters and return values. Functions are first-class objects in Python.

Syntax

```
def function_name(parameters):
    """Optional docstring."""
    ...
    return value
```

Example

```
def add(a, b):
    return a + b

result = add(3, 4)
print(result)
```

Interview focus: Difference between print and return; functions are objects; default return value is None.

Industry use: Core unit of decomposition, reuse, testing and maintainability.

Common trap: Avoid functions that do many unrelated things; small cohesive functions are easier to test.

Function Arguments

Priority: ★★★ | MUST KNOW

Python supports positional, keyword, default, variable-length, positional-only and keyword-only arguments.

Syntax

```
def f(positional, default=1, *args, keyword_only=0, **kwargs):
    ...
```

Example

```
def create_user(name, age=18, *, active=True):
    return {"name": name, "age": age, "active": active}

create_user("Riya")
create_user("Riya", 25, active=False)
```

Interview focus: Argument order, default values, keyword-only parameters, mutable-default trap.

Industry use: Stable APIs and readable function calls depend on good parameter design.

Common trap: Never use a mutable object like [] or {} as a default when you intend a fresh object per call.

Mutable Default Argument Trap

Priority: ★★★ | MUST KNOW

Default argument expressions are evaluated once when the function is defined, not every time it is called.

Syntax

```
def f(items=None):
    if items is None:
        items = []
```

Example

```
def bad_add(x, bucket=[]):
    bucket.append(x)
    return bucket

print(bad_add(1)) # [1]
print(bad_add(2)) # [1, 2]

def good_add(x, bucket=None):
    if bucket is None:
        bucket = []
    bucket.append(x)
    return bucket
```

Interview focus: Very common interview trap: why does a list default remember previous calls?

Industry use: Avoids hidden shared state bugs in services and libraries.

Lambda Functions

Priority: ★★ | IMPORTANT

lambda creates a small anonymous function containing one expression.

Syntax

```
lambda arguments: expression
```

Example

```
square = lambda x: x * x
print(square(5))

users = [("A", 30), ("B", 20)]
users.sort(key=lambda item: item[1])
```

Interview focus: Lambda is an expression, not a full replacement for def; often used as key functions.

Industry use: Sorting, callbacks, map/filter pipelines.

Common trap: If a lambda is complex, use def and give the behavior a meaningful name.

Recursion

Priority: ★★ | IMPORTANT

A recursive function calls itself and needs a base case to stop. Python has a recursion depth limit, so iteration is often better for large inputs.

Syntax

```
def recursive(x):
    if base_case:
        return result
    return recursive(smaller_problem)
```

Example

```
def factorial(n):
    if n <= 1:
        return 1
    return n * factorial(n - 1)

print(factorial(5)) # 120
```

Interview focus: Base case; call stack; recursion vs iteration; recursion depth.

Industry use: Useful for tree traversal and naturally recursive structures.

Common trap: Missing or incorrect base cases cause infinite recursion/RecursionError.

Modules and Imports

Priority: ★★★ | MUST KNOW

A module is a Python file containing reusable code. Importing allows functions, classes and variables to be organized across files.

Syntax

```
import module
from module import name
import module as alias
```

Example

```
import math
from pathlib import Path

print(math.sqrt(16))
print(Path.cwd())
```

Interview focus: `import x` vs `from x import y`; name collisions; `if \_\_name\_\_ == "\_\_main\_\_":`.

Industry use: Every non-trivial project is split into modules/packages.

Common trap: Avoid `from module import \*` in production code.

requests Module

Priority: ★★★ | MUST KNOW

`requests` is a popular third-party HTTP client. It is not part of the Python standard library and normally must be installed with pip.

Syntax

```
pip install requests

import requests
response = requests.get(url, timeout=10)
```

Example

```
import requests
```

```
url = "https://api.example.com/users"
try:
    r = requests.get(url, timeout=10)
    r.raise_for_status()
    data = r.json()
except requests.RequestException as exc:
    print("HTTP error:", exc)
```

Interview focus: Use timeouts; `raise_for_status()`; JSON decoding; exception handling; query params and headers.

Industry use: HTTP APIs are everywhere in backend, automation, data and integration work.

Common trap: Do not make network calls without a timeout in production code.

Function Scope and Lifetime (LEGB)

Priority: ★★★ | MUST KNOW

Name lookup generally follows LEGB: Local, Enclosing, Global, Built-in. Local variables normally exist while the function call is active, though referenced objects can live longer.

Syntax

```
Local -> Enclosing -> Global -> Built-in
```

Example

```
x = "global"

def outer():
    x = "enclosing"
    def inner():
        x = "local"
        print(x)
    inner()

outer()
```

Interview focus: LEGB rule; closure basics; local variable lifetime.

Industry use: Understanding scope prevents shadowing bugs and makes closures/decorators easier.

Common trap: Assigning to a name inside a function makes it local unless declared `global/nonlocal`.

global and nonlocal

Priority: ★★ | IMPORTANT

`'global'` allows assignment to a module-level name. `'nonlocal'` allows assignment to a name in an enclosing function scope.

Syntax

```
global name
nonlocal name
```

Example

```
count = 0

def increment():
    global count
    count += 1

def outer():
    x = 0
    def inner():
        nonlocal x
        x += 1
    inner()
    return x
```

Interview focus: global vs nonlocal and why excessive global state is harmful.

Industry use: Occasionally needed for simple module state or closures, but dependency injection/state objects are often cleaner.

Common trap: Overusing global variables makes code hard to test and reason about.

Docstrings

Priority: ★★ | IMPORTANT

A docstring is a string literal placed first in a module, class, method or function to document its public behavior. Tools can read it through `__doc__` and `help()`.

Syntax

```
def f(x):
    """Return a processed value for x."""
    ...
```

Example

```
def divide(a, b):
    """Return a / b.

    Raises:
        ZeroDivisionError: if b is zero.
    """
    return a / b
```

Interview focus: Docstring vs comment; common styles such as Google/NumPy/Sphinx.

Industry use: API documentation, IDE hints, onboarding and maintainability.

7. Lists

List Basics

Priority: ★★★ | MUST KNOW

A list is an ordered, mutable sequence that can hold mixed object types and duplicate values.

Syntax

```
items = [value1, value2, ...]
```

Example

```
nums = [10, 20, 30]
nums[0] = 99
nums.append(40)
print(nums)
```

Interview focus: Mutable vs immutable; indexing/slicing; shallow copy vs aliasing.

Industry use: Most common general-purpose collection for ordered data.

Common trap: `b = a` does not copy a list; both names point to the same object.

List Methods

Priority: ★★★ | MUST KNOW

Important list methods modify or inspect a list.

Syntax

```
append(x)  extend(iterable)  insert(i, x)
remove(x)  pop([i])  clear()
index(x)   count(x)
sort(key=..., reverse=...)  reverse()  copy()
```

Example

```
items = [3, 1, 2]
items.append(4)
items.extend([5, 6])
last = items.pop()
items.sort()
print(items, last)
```

Interview focus: `append` vs `extend`; `remove` vs `pop`; `list.sort()` vs `sorted()`.

Industry use: Data preparation, queues/stacks (small workloads), ordered records.

Common trap: `list.sort()` returns None because it sorts in place. `sorted()` returns a new list.

List Comprehension

Priority: ★★★ | MUST KNOW

A list comprehension builds a list concisely from an iterable, optionally filtering items.

Syntax

```
[expression for item in iterable if condition]
```

Example

```
squares = [n * n for n in range(6)]
evens = [n for n in range(10) if n % 2 == 0]
words = [name.upper() for name in ["riya", "aman"]]
```

Interview focus: Translate between a normal loop and a comprehension; nested comprehensions; readability.

Industry use: Very common in data transformation and clean Python code.

Common trap: Do not cram complex business logic into a comprehension.

8. Tuples

Tuple Basics

Priority: ★★ | IMPORTANT

A tuple is an ordered, immutable sequence. It can contain mutable objects, but the tuple structure itself cannot be changed.

Syntax

```
point = (10, 20)
single = (5,)
```

Example

```
point = (10, 20)
print(point[0])
coordinates = 10, 20 # parentheses optional in many cases
```

Interview focus: Why `(5,)` needs a comma; tuples can be dictionary keys if all contained values are hashable.

Industry use: Fixed records, multiple return values, dictionary keys, unpacking.

Tuple Unpacking

Priority: ★★★ | MUST KNOW

Unpacking assigns items from an iterable to multiple variables. Starred unpacking captures remaining items.

Syntax

```
a, b = iterable
first, *middle, last = iterable
```

Example

```
name, age = ("Riya", 25)
first, *rest = [10, 20, 30, 40]
a, b = b, a
```

Interview focus: Swapping without a temporary variable; extended unpacking.

Industry use: Cleanly deconstructing function returns and records.

Tuple Methods

Priority: ★ | USEFUL

Tuples have only two main methods because they are immutable: `count()` and `index()`.

Syntax

```
t.count(value)
t.index(value)
```

Example

```
t = (1, 2, 2, 3)
print(t.count(2))
print(t.index(3))
```

9. Sets

Set Basics

Priority: ★★★ | MUST KNOW

A set is an unordered collection of unique, hashable elements. Membership testing is typically very fast.

Syntax

```
s = {1, 2, 3}
empty = set()
```

Example

```
tags = {"python", "api", "python"}
print(tags)    # duplicate removed
print("api" in tags)
```

Interview focus: Why `{}` is an empty dict, not an empty set; uniqueness and hashability.

Industry use: Deduplication, membership checks, comparing groups.

Common trap: Sets do not support indexing and should not be relied on for a stable display order.

Set Methods

Priority: ★★ | IMPORTANT

Common mutating methods include add, update, remove, discard, pop and clear.

Syntax

```
add(x)  update(iterable)
remove(x)  discard(x)  pop()  clear()
copy()
```

Example

```
s = {1, 2}
s.add(3)
s.update([3, 4, 5])
s.discard(99)  # no error if missing
print(s)
```

Interview focus: remove raises KeyError if missing; discard does not.

Set Operations

Priority: ★★★ | MUST KNOW

Set algebra provides union, intersection, difference and symmetric difference.

Syntax

```
a | b    # union
a & b    # intersection
a - b    # difference
a ^ b    # symmetric difference
a <= b   # subset
```

Example

```
a = {1, 2, 3}
b = {3, 4}
print(a | b)
print(a & b)
print(a - b)
```

```
print(a ^ b)
```

Interview focus: Know union/intersection/difference and subset/superset checks.

Industry use: Permissions, feature flags, deduping IDs, comparing datasets.

10. Dictionaries

Dictionary Basics

Priority: ★★★ | **MUST KNOW**

A dictionary stores key-value pairs. Keys must be hashable; values can be any type. Modern Python preserves insertion order.

Syntax

```
d = {key: value, ...}
```

Example

```
user = {"name": "Riya", "age": 25}
print(user["name"])
user["city"] = "Delhi"
user["age"] = 26
```

Interview focus: KeyError; hashable keys; insertion order; nested dictionaries.

Industry use: JSON-like data, configuration, counters, lookup tables and API payloads.

Common trap: Use `.get()` when a missing key is expected and you want a default instead of KeyError.

Dictionary Methods

Priority: ★★★ | **MUST KNOW**

Core methods support safe lookup, iteration, updates and removal.

Syntax

```
get(key, default)
keys() values() items()
update(other)
pop(key[, default]) popitem()
setdefault(key, default)
copy() clear()
```

Example

```
user = {"name": "Riya", "age": 25}
print(user.get("city", "Unknown"))
for key, value in user.items():
    print(key, value)
user.update({"age": 26, "active": True})
```

Interview focus: `.get()` vs indexing; `.items()` unpacking; `setdefault` behavior.

Industry use: One of the most heavily used structures in real Python applications.

Common trap: Views returned by `keys()`/`values()`/`items()` are dynamic views, not independent lists.

Dictionary Comprehension

Priority: ★★★ | **MUST KNOW**

Dictionary comprehensions create dictionaries from iterables using key:value expressions.

Syntax

```
{key_expr: value_expr for item in iterable if condition}
```

Example

```
squares = {n: n * n for n in range(5)}
active = {k: v for k, v in {"a": 1, "b": 0}.items() if v}
print(squares)
```

Interview focus: Frequently paired with `.items()` and filtering.

Industry use: Transforms API/configuration data cleanly.

11. Object-Oriented Programming (OOP) - IMPORTANT

High-frequency interview area. Understand concepts, not just syntax.

OOP mental model: A class is a blueprint/type; an object is an instance of that class. Attributes store state; methods define behavior. Use OOP when state and behavior naturally belong together.

Class and Objects

Priority: ★★★ | MUST KNOW

A class defines attributes and methods. Calling the class normally creates an object (instance).

Syntax

```
class ClassName:
    ...

obj = ClassName()
```

Example

```
class Car:
    def drive(self):
        return "Driving"

car1 = Car()
print(car1.drive())
```

Interview focus: Class vs object vs instance; methods are functions defined in a class body.

Industry use: Domain models, services, reusable components and framework code.

self Keyword

Priority: ★★★ | MUST KNOW

``self`` conventionally refers to the current instance. Python passes the instance automatically when an instance method is called through an object.

Syntax

```
class User:
    def method(self, ...):
        self.attribute = value
```

Example

```
class User:
    def set_name(self, name):
        self.name = name

u = User()
u.set_name("Riya")
print(u.name)
```

Interview focus: ``self`` is not a reserved keyword, but the convention should be followed. Why `u.set_name("Riya")`` maps to `User.set_name(u, "Riya")``.

Industry use: Central to Python OOP and debugging method calls.

Constructor: `__init__`

Priority: ★★★ | MUST KNOW

`__init__`` initializes a newly created instance after object creation. It is commonly called the constructor in interviews, though technically `__new__`` creates the object and `__init__`` initializes it.

Syntax

```
class ClassName:
```

```
def __init__(self, ...):
    self.attribute = value
```

Example

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

e = Employee("Aman", 80000)
```

Interview focus: Difference between `__new__` and `__init__` at a high level; instance attributes usually created in `__init__`.

Industry use: Initialization and validation of object state.

Instance Attributes vs Class Attributes

Priority: ★★★ | MUST KNOW

Instance attributes belong to each object. Class attributes are stored on the class and shared as defaults across instances unless shadowed.

Syntax

```
class Example:
    class_attr = value
    def __init__(self):
        self.instance_attr = value
```

Example

```
class Employee:
    company = "ABC Ltd"
    def __init__(self, name):
        self.name = name

e1 = Employee("A")
e2 = Employee("B")
print(e1.company, e2.company)
```

Interview focus: Lookup order; shared mutable class attributes can be dangerous.

Industry use: Class-level configuration/counters vs per-object state.

Common trap: Do not put a mutable list/dict at class level if each instance should have its own independent collection.

Instance, Class and Static Methods

Priority: ★★★ | MUST KNOW

Instance methods receive ``self``; class methods receive ``cls`` and use `@classmethod`; static methods receive neither automatically and use `@staticmethod`.

Syntax

```
class C:
    def instance_method(self): ...

    @classmethod
    def class_method(cls): ...

    @staticmethod
    def static_method(...): ...
```

Example

```
class Temperature:
    scale = "C"

    def __init__(self, value):
        self.value = value

    @classmethod
    def from_fahrenheit(cls, f):
        return cls((f - 32) * 5 / 9)
```

```
@staticmethod
def is_freezing(c):
    return c <= 0
```

Interview focus: When to use each; class methods as alternative constructors.

Industry use: Very common API design pattern in libraries and frameworks.

Four Pillars of OOP

Priority: ★★★ | MUST KNOW

The four commonly discussed pillars are encapsulation, abstraction, inheritance and polymorphism.

Syntax

```
Encapsulation: bundle state + behavior
Abstraction: expose essential interface, hide details
Inheritance: derive behavior from a base class
Polymorphism: same interface, different implementations
```

Example

```
class Payment:
    def pay(self):
        raise NotImplementedError

class CardPayment(Payment):
    def pay(self):
        return "Paid by card"
```

Interview focus: Explain each pillar with a Python example. Python relies more on conventions and protocols than strict private access.

Industry use: Helps model replaceable components and interfaces.

Common trap: Do not force inheritance where composition is simpler.

Encapsulation and Naming Conventions

Priority: ★★ | IMPORTANT

Python has no strict private keyword. `\_name` means internal by convention; `\_\_name` triggers name mangling.

Syntax

```
self.public
self._internal
self.__mangled
```

Example

```
class Account:
    def __init__(self, balance):
        self._balance = balance

    def deposit(self, amount):
        self._balance += amount
```

Interview focus: Single underscore convention vs double-underscore name mangling.

Industry use: Libraries use these conventions to communicate public vs internal APIs.

Inheritance

Priority: ★★★ | MUST KNOW

Inheritance lets a subclass reuse or extend behavior from a base class.

Syntax

```
class Child(Parent):
    ...
```

Example

```
class Animal:
```

```
def speak(self):
    return "sound"

class Dog(Animal):
    def speak(self):
        return "bark"

print(Dog().speak())
```

Interview focus: Method overriding; `isinstance`; multiple inheritance/MRO at a high level.

Industry use: Framework extension points and reusable base behavior.

super().method_name()

Priority: ★★★ | **MUST KNOW**

`super()` provides access to methods following the method resolution order (MRO), commonly to call a parent implementation.

Syntax

```
super().__init__(...)
super().method_name(...)
```

Example

```
class Employee:
    def __init__(self, name):
        self.name = name

class Manager(Employee):
    def __init__(self, name, team_size):
        super().__init__(name)
        self.team_size = team_size
```

Interview focus: super is MRO-aware and is important in cooperative multiple inheritance.

Industry use: Safely extending parent initialization/behavior.

Common trap: Hard-coding `Parent.method(self)` can break cooperative multiple inheritance patterns.

Polymorphism and Duck Typing

Priority: ★★★ | **MUST KNOW**

Polymorphism means different objects can respond to the same operation. Python often uses duck typing: if an object supports the required protocol, its exact class may not matter.

Syntax

```
def make_sound(obj):
    return obj.speak()
```

Example

```
class Dog:
    def speak(self): return "bark"
class Cat:
    def speak(self): return "meow"

for animal in [Dog(), Cat()]:
    print(animal.speak())
```

Interview focus: Duck typing; method overriding; protocols/interfaces concept.

Industry use: Flexible plug-in architectures and test doubles.

Operator Overloading

Priority: ★★ | **IMPORTANT**

Special (dunder) methods let custom objects work with operators such as +, == and <.

Syntax

```
__add__ __sub__ __mul__
__eq__ __lt__ __le__
```

Example

```
class Vector:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)

    def __repr__(self):
        return f"Vector({self.x}, {self.y})"

print(Vector(1,2) + Vector(3,4))
```

Interview focus: Operator overloading should preserve intuitive meaning.

Industry use: Numeric/data model classes and domain-specific types.

Function Inside a Class vs Outside a Class

Priority: ★★★ | MUST KNOW

A function defined in a class body becomes a function descriptor and is typically used as a method. When accessed through an instance, Python binds the instance to it. A standalone function has no automatic object binding.

Syntax

```
def standalone(x): ...

class C:
    def method(self, x): ...
```

Example

```
def greet(name):
    return f"Hi {name}"

class User:
    def __init__(self, name):
        self.name = name
    def greet(self):
        return f"Hi {self.name}"
```

Interview focus: Use a method when behavior depends on or conceptually belongs to object/class state. Use a standalone function for independent behavior.

Industry use: Good design avoids unnecessary classes and unnecessary global helper logic.

Composition vs Inheritance

Priority: ★★★ | MUST KNOW

Composition builds objects using other objects; inheritance models an "is-a" relationship. Composition is often more flexible and less tightly coupled.

Syntax

```
class Engine: ...
class Car:
    def __init__(self, engine):
        self.engine = engine
```

Example

```
class Logger:
    def log(self, msg): print(msg)

class Service:
    def __init__(self, logger):
        self.logger = logger
```

Interview focus: Interviewers often ask when composition is better than inheritance.

Industry use: Dependency injection, testability and modular design.

12. Advanced Python Concepts

Decorators

Priority: ★★★ | MUST KNOW

A decorator takes a callable and returns a replacement callable, allowing behavior to be added without changing the original function body.

Syntax

```
@decorator
def function(...):
    ...
```

Example

```
from functools import wraps

def log_call(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        print("Calling", func.__name__)
        return func(*args, **kwargs)
    return wrapper

@log_call
def add(a, b):
    return a + b
```

Interview focus: Functions are first-class; closures; why `functools.wraps` matters.

Industry use: Authentication, caching, logging, retries, metrics, framework routing.

Common trap: A decorator must return the wrapper/function. Forgetting this makes the decorated name become None.

Decorator with Arguments

Priority: ★★★ | MUST KNOW

A decorator that accepts arguments usually has three nested levels: decorator factory -> decorator -> wrapper.

Syntax

```
def decorator_factory(option):
    def decorator(func):
        def wrapper(*args, **kwargs):
            ...
        return wrapper
    return decorator
```

Example

```
from functools import wraps

def repeat(times):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            result = None
            for _ in range(times):
                result = func(*args, **kwargs)
            return result
        return wrapper
    return decorator

@repeat(3)
def hello():
    print("Hello")
```

Interview focus: Be able to explain why three layers are needed.

Industry use: Configurable retries, permissions, rate limits and instrumentation.

Getters, Setters and @property

Priority: ★★★ | MUST KNOW

`@property` provides attribute-style access while running methods for validation or computed values. A setter can validate assignments.

Syntax

```
@property
def name(self): ...

@name.setter
def name(self, value): ...
```

Example

```
class Product:
    def __init__(self, price):
        self.price = price

    @property
    def price(self):
        return self._price

    @price.setter
    def price(self, value):
        if value < 0:
            raise ValueError("price cannot be negative")
        self._price = value
```

Interview focus: Prefer properties over Java-style `get_x/set_x` when attribute syntax is natural.

Industry use: Validation while keeping a stable public API.

Dunder / Magic Methods

Priority: ★★★ | MUST KNOW

Dunder methods such as `__init__`, `__repr__`, `__len__` and `__iter__` implement Python data-model protocols.

Syntax

```
__init__  __repr__  __str__
__len__   __iter__  __next__
__enter__ __exit__
__eq__    __hash__  __call__
```

Example

```
class Box:
    def __init__(self, items):
        self.items = items
    def __len__(self):
        return len(self.items)
    def __repr__(self):
        return f"Box({self.items!r})"

b = Box([1,2,3])
print(len(b), b)
```

Interview focus: Know `__repr__` vs `__str__`, and how protocols enable built-in syntax.

Industry use: Well-designed domain types integrate naturally with Python.

Errors and Exception Hierarchy

Priority: ★★★ | MUST KNOW

Exceptions represent runtime problems. Common built-ins include `ValueError`, `TypeError`, `KeyError`, `IndexError`, `AttributeError`, `FileNotFoundError` and `ZeroDivisionError`.

Syntax

```
raise ExceptionType("message")
```

Example

```
def set_age(age):
    if age < 0:
        raise ValueError("age must be non-negative")
```

Interview focus: Syntax errors vs exceptions; choosing specific exception types; custom exceptions.

Industry use: Clear failure modes improve reliability and debugging.

try / except / else / finally

Priority: ★★★ | **MUST KNOW**

Use try/except to handle expected failures. `else` runs only if no exception occurred; `finally` runs whether an exception happened or not.

Syntax

```
try:
    risky()
except SpecificError as exc:
    handle(exc)
else:
    success_only()
finally:
    cleanup()
```

Example

```
try:
    number = int("42")
except ValueError as exc:
    print("Bad input", exc)
else:
    print("Converted", number)
finally:
    print("Done")
```

Interview focus: Catch specific exceptions; finally semantics; re-raising with `raise`.

Industry use: API calls, file access, database code and validation all need disciplined exception handling.

Common trap: Avoid `except Exception: pass` because it hides real bugs and makes systems hard to diagnose.

map

Priority: ★★ | **IMPORTANT**

map applies a function to each item and returns a lazy iterator in Python 3.

Syntax

```
map(function, iterable)
```

Example

```
nums = [1, 2, 3]
doubled = list(map(lambda x: x * 2, nums))
print(doubled)
```

Interview focus: map is lazy; list comprehensions are often more readable for simple transformations.

Industry use: Functional pipelines and existing callable reuse.

filter

Priority: ★★ | **IMPORTANT**

filter keeps items for which a predicate is truthy and returns a lazy iterator.

Syntax

```
filter(function, iterable)
```

Example

```
nums = [1, 2, 3, 4]
evens = list(filter(lambda x: x % 2 == 0, nums))
```

Interview focus: Compare filter with a list comprehension.

Industry use: Filtering pipelines.

reduce

Priority: ★★ | IMPORTANT

functools.reduce repeatedly combines items into one accumulated value.

Syntax

```
from functools import reduce
reduce(function, iterable[, initial])
```

Example

```
from functools import reduce
nums = [1, 2, 3, 4]
product = reduce(lambda a, b: a * b, nums, 1)
print(product)
```

Interview focus: Understand the accumulator idea; built-ins such as sum/min/max are clearer when applicable.

Industry use: Some functional aggregation tasks.

Walrus Operator :=

Priority: ★★ | IMPORTANT

The assignment expression assigns a value and returns it as part of a larger expression. Available since Python 3.8.

Syntax

```
name := expression
```

Example

```
while (line := input("Text: ")) != "quit":
    print(line)

data = [1, 2, 3, 4, 5]
if (n := len(data)) > 3:
    print(f"Large list: {n}")
```

Interview focus: Use it to avoid repeated work, but prioritize readability.

Industry use: Useful in loops, regex matching and expensive expressions.

*args

Priority: ★★★ | MUST KNOW

`\*args` collects extra positional arguments into a tuple. In a call, `\*iterable` unpacks positional arguments.

Syntax

```
def f(*args): ...
f(*values)
```

Example

```
def total(*numbers):
    return sum(numbers)

print(total(1, 2, 3))
values = [4, 5]
print(total(*values))
```

Interview focus: Collection vs unpacking behavior; args is a convention, the * is what matters.

Industry use: Flexible APIs, wrappers and decorators.

**kwargs

Priority: ★★★ | MUST KNOW

`\*\*kwargs` collects extra keyword arguments into a dictionary. In a call, `\*\*mapping` unpacks keyword arguments.

Syntax

```
def f(**kwargs): ...
f(**mapping)
```

Example

```
def show(**info):
    for key, value in info.items():
        print(key, value)

show(name="Riya", age=25)
data = {"city": "Delhi", "active": True}
show(**data)
```

Interview focus: kwargs is a dict; duplicate keyword names during unpacking raise `TypeError`.

Industry use: Configuration layers, framework hooks and wrappers.

Combined `*args` and `**kwargs`

Priority: ★★★ | MUST KNOW

Many wrappers accept both to forward arbitrary positional and keyword arguments.

Syntax

```
def wrapper(*args, **kwargs):
    return target(*args, **kwargs)
```

Example

```
def debug_call(func, *args, **kwargs):
    print("args:", args)
    print("kwargs:", kwargs)
    return func(*args, **kwargs)

def greet(name, greeting="Hi"):
    return f"{greeting} {name}"

print(debug_call(greet, "Aman", greeting="Hello"))
```

Interview focus: Canonical parameter order: positional-only, normal/default, `*args`, keyword-only, `**kwargs`.

Industry use: Essential for decorators, adapters and generic APIs.

13. Files, OS, shutil and Command-Line Arguments

Opening and Reading Files

Priority: ★★★ | MUST KNOW

`open()` returns a file object. Common modes include `r` (read), `w` (write/truncate), `a` (append), `x` (create), `b` (binary), and `+` (read/write combination).

Syntax

```
open(path, mode="r", encoding="utf-8")
```

Example

```
f = open("notes.txt", "r", encoding="utf-8")
try:
    content = f.read()
finally:
    f.close()
```

Interview focus: Text vs binary mode; encoding; why context managers are preferred.

Industry use: Config files, logs, CSV/text processing, exports.

Common trap: Always specify encoding for portable text-file behavior when practical.

with Context Manager

Priority: ★★★ | MUST KNOW

`with` automatically performs cleanup by calling the context manager protocol. For files it closes the file even if an exception occurs.

Syntax

```
with open(path, mode, encoding="utf-8") as f:
    ...
```

Example

```
with open("notes.txt", "r", encoding="utf-8") as f:
    content = f.read()

print(f.closed) # True
```

Interview focus: How `with` relates to `__enter__`/`__exit__`; preferred file pattern.

Industry use: Reliable resource management for files, locks, DB connections and more.

read, readline, readlines

Priority: ★★ | IMPORTANT

Different read methods support whole-file, one-line and list-of-lines access.

Syntax

```
f.read([size])
f.readline()
f.readlines()
```

Example

```
with open("data.txt", encoding="utf-8") as f:
    first = f.readline()

with open("data.txt", encoding="utf-8") as f:
    lines = f.readlines()
```

Interview focus: For large files, direct iteration is usually more memory-efficient than `readlines()`.

Line-by-Line File Processing

Priority: ★★★ | MUST KNOW

File objects are iterable, so iterate directly for memory-efficient processing.

Syntax

```
with open(path) as f:
    for line in f:
        ...
```

Example

```
with open("access.log", encoding="utf-8") as f:
    for line_no, line in enumerate(f, 1):
        line = line.rstrip("\n")
        if "ERROR" in line:
            print(line_no, line)
```

Interview focus: Best practice for large text/log files.

Industry use: Log parsing, ETL, data cleanup.

Writing Files

Priority: ★★★ | MUST KNOW

Mode `w` writes from the beginning and truncates an existing file. `write()` returns the number of characters written.

Syntax

```
with open(path, "w", encoding="utf-8") as f:
    f.write(text)
```

Example

```
with open("report.txt", "w", encoding="utf-8") as f:
    f.write("Python Report\n")
    f.writelines(["A\n", "B\n"])
```

Interview focus: writelines does not automatically add newline characters.

Industry use: Generating reports, exports and config files.

Common trap: Opening an existing file with w destroys its previous contents.

Appending Files

Priority: ★★ | **IMPORTANT**

Mode `a` writes at the end of the file, creating it if needed.

Syntax

```
with open(path, "a", encoding="utf-8") as f:
    f.write(text)
```

Example

```
with open("events.log", "a", encoding="utf-8") as f:
    f.write("service started\n")
```

Interview focus: Difference between write mode and append mode.

Industry use: Simple audit/event logs and accumulating output.

os Module

Priority: ★★★ | **MUST KNOW**

The standard-library os module provides operating-system interfaces such as environment variables, paths, directories and process-related utilities. Prefer pathlib for many path tasks, but os remains common.

Syntax

```
import os
os.getcwd()
os.listdir(path)
os.makedirs(path, exist_ok=True)
os.environ.get("NAME")
```

Example

```
import os

print(os.getcwd())
os.makedirs("output", exist_ok=True)
api_key = os.environ.get("API_KEY")
```

Interview focus: Environment variables and cross-platform path concerns.

Industry use: Configuration, deployment scripts, filesystem automation.

Common trap: Do not hard-code secrets in source code; environment variables are one common configuration mechanism.

shutil Module

Priority: ★★ | **IMPORTANT**

shutil provides higher-level file operations such as copying, moving and removing directory trees.

Syntax

```
shutil.copy(src, dst)
shutil.copy2(src, dst)
shutil.move(src, dst)
shutil.rmtree(path)
```

Example

```
import shutil

shutil.copy2("config.ini", "backup/config.ini")
# shutil.rmtree("temp") # destructive: use carefully
```

Interview focus: copy vs copy2; destructive operations need care.

Industry use: Deployment, backup, build and automation scripts.

argparse

Priority: ★★★ | MUST KNOW

argparse builds command-line interfaces using the standard library. It handles parsing, help text, types, defaults and flags.

Syntax

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument(...)
args = parser.parse_args()
```

Example

```
import argparse

parser = argparse.ArgumentParser(description="Process a file")
parser.add_argument("path")
parser.add_argument("--limit", type=int, default=10)
parser.add_argument("--verbose", action="store_true")
args = parser.parse_args()

print(args.path, args.limit, args.verbose)
```

Interview focus: Positional vs optional arguments; type conversion; store_true.

Industry use: Production scripts, data jobs, admin tools and utilities.

14. External Modules and Package Management

pip and Third-Party Packages

Priority: ★★★ | MUST KNOW

Third-party packages are installed separately from the standard library, usually with pip inside a virtual environment.

Syntax

```
python -m pip install package_name
python -m pip show package_name
python -m pip freeze
```

Example

```
# Example shell commands
python -m venv .venv
# activate the environment, then:
python -m pip install requests
```

Interview focus: Standard library vs third-party package; why `python -m pip` can be safer than bare pip.

Industry use: Dependency management is essential in real projects.

Common trap: Do not install every project dependency globally; use isolated environments.

Useful External Modules to Recognize

Priority: ★★ | IMPORTANT

The exact stack depends on the role. Common names include requests/httpx (HTTP), pandas/numpy (data), pytest (testing), FastAPI/Django/Flask (web), SQLAlchemy (database ORM), and rich (CLI output).

Syntax

```
import requests
import pandas as pd
import pytest
```

Example

```
import requests
```

```
r = requests.get("https://example.com", timeout=5)
print(r.status_code)
```

Interview focus: Interview expectations depend on role. Strong fundamentals matter more than memorizing package lists.

Industry use: Recognizing common ecosystem tools helps you navigate codebases quickly.

15. Threading and Concurrency

threading Basics

Priority: ★★★ | MUST KNOW

A thread is an execution path within a process. Python threads are useful mainly for I/O-bound concurrency such as network calls or waiting on files.

Syntax

```
from threading import Thread
thread = Thread(target=function, args=(...))
thread.start()
thread.join()
```

Example

```
from threading import Thread
import time

def worker(name):
    time.sleep(1)
    print("done", name)

threads = [Thread(target=worker, args=(i,)) for i in range(3)]
for t in threads:
    t.start()
for t in threads:
    t.join()
```

Interview focus: start vs join; shared memory; race conditions; daemon threads; I/O-bound vs CPU-bound.

Industry use: Parallel waiting for APIs/files can improve throughput.

GIL and When Threading Helps

Priority: ★★★ | MUST KNOW

In the standard CPython build commonly encountered today, the Global Interpreter Lock (GIL) means only one thread executes Python bytecode at a time, so threads usually do not speed up CPU-bound pure-Python work. They can still be effective for I/O-bound work because the GIL is released while waiting in many I/O operations.

Syntax

```
I/O-bound -> threading can help
CPU-bound -> often use multiprocessing / native vectorized code
```

Example

```
Typical I/O-bound tasks:
- HTTP requests
- file/network waiting
- database calls

Typical CPU-bound tasks:
- heavy Python loops
- image/math work without native parallelism
```

Interview focus: Classic interview question: explain the GIL and why threading can still help I/O-bound workloads.

Industry use: Choosing the correct concurrency model avoids wasting complexity.

Race Conditions and Lock

Priority: ★★★ | MUST KNOW

When threads share mutable state, operations can interleave unpredictably. A Lock protects a critical section so only one thread enters it at a time.

Syntax

```
from threading import Lock
lock = Lock()

with lock:
    # critical section
```

Example

```
from threading import Lock

lock = Lock()
balance = 0

def add_money(amount):
    global balance
    with lock:
        balance += amount
```

Interview focus: Race condition, critical section, lock/deadlock basics.

Industry use: Shared-state services, workers and caches.

Common trap: Locks solve some races but can create deadlocks or performance bottlenecks if used badly.

ThreadPoolExecutor

Priority: ★★★ | MUST KNOW

`concurrent.futures.ThreadPoolExecutor` is often easier and safer than manually creating many Thread objects.

Syntax

```
from concurrent.futures import ThreadPoolExecutor
with ThreadPoolExecutor(max_workers=N) as ex:
    results = ex.map(func, items)
```

Example

```
from concurrent.futures import ThreadPoolExecutor

def fetch(url):
    return f"fetch {url}"

urls = ["a", "b", "c"]
with ThreadPoolExecutor(max_workers=3) as executor:
    for result in executor.map(fetch, urls):
        print(result)
```

Interview focus: High-level concurrency API; futures; exception propagation.

Industry use: Common for bounded pools of I/O-bound work.

16. High-Value Interview Traps and Rapid Revision

Memorize these distinctions: `==` value equality vs `is` identity; list mutable vs tuple immutable; append vs extend; remove vs pop; sort vs sorted; shallow copy vs alias; class vs instance attribute; instance/class/static method; break vs continue vs pass; global vs nonlocal; iterable vs iterator; exception handling vs hiding errors.

Question	Short interview answer
Why is `is` not the same as `==`?	`==` asks whether values are equal; `is` asks whether both references point to the same object. Use `is None` for the singleton None.
What is mutable vs immutable?	Mutable objects can change in place (list, dict, set). Immutable objects cannot change in place (int, float, str, tuple, frozenset).
What is the mutable default argument problem?	Defaults are evaluated once at function definition time, so a mutable default can be shared across calls.
Difference between append and extend?	append adds one object as a single element; extend iterates over its argument and adds each element.
Difference between sort and sorted?	list.sort mutates a list and returns None; sorted accepts any iterable and

	returns a new list.
What is a shallow copy?	A new outer container containing references to the same nested objects. Use <code>copy.deepcopy</code> when truly independent nested objects are required.
What does <code>`self`</code> mean?	The current instance passed to an instance method. It is conventional, not a special reserved word.
What does <code>`@classmethod`</code> receive?	The class as <code>`cls`</code> , useful for class-level behavior and alternative constructors.
What does <code>`@staticmethod`</code> receive automatically?	Neither <code>self</code> nor <code>cls</code> . It is a namespaced utility related to the class conceptually.
What is a decorator?	A callable that takes a callable and returns a replacement callable, commonly adding behavior around the original function.
Why use <code>`with open(...)`</code>?	It guarantees resource cleanup using the context manager protocol, even when exceptions occur.
What is LEGB?	Local, Enclosing, Global, Built-in name lookup order.
When is threading useful in Python?	Usually I/O-bound concurrency; CPU-bound pure-Python work often needs multiprocessing or native code because of the CPython GIL.
What does <code>`finally`</code> do?	Runs whether an exception occurs or not, making it suitable for cleanup.
What is duck typing?	Behavior matters more than exact type: if an object supports the expected operations/protocol, it can be used.

Mini coding patterns worth practicing

```
# 1) Frequency counter
counts = {}
for item in items:
    counts[item] = counts.get(item, 0) + 1

# 2) Deduplicate while keeping order
unique = list(dict.fromkeys(items))

# 3) Safe dictionary lookup
value = config.get("timeout", 30)

# 4) Enumerate with index
for i, item in enumerate(items, start=1):
    ...

# 5) Pair sequences
for left, right in zip(a, b):
    ...

# 6) Sort by a field
users = sorted(users, key=lambda u: u["age"])

# 7) Context-managed file
with open(path, encoding="utf-8") as f:
    for line in f:
        ...
```

17. One-Page Final Revision Checklist

- ☐ Can I explain variables as names bound to objects and Python as dynamically typed?
- ☐ Can I convert input safely using `int/float/str` and handle invalid conversion?
- ☐ Do I know ``==`` vs ``is``, ``in``, logical short-circuiting and truthy/falsy values?
- ☐ Can I write `if/elif/else`, `match/case`, `for/while`, `break/continue/pass` and `loop-else`?
- ☐ Can I index/slice strings and use `split/join/strip/replace/f-strings`?
- ☐ Can I explain functions, return, argument types, lambda, recursion, LEGB, global/nonlocal and docstrings?
- ☐ Do I know list methods, list comprehension, tuple unpacking, set operations and dictionary methods?
- ☐ Can I explain class/object, `self`, `__init__`, instance vs class attributes, `classmethod`, `staticmethod` and inheritance?
- ☐ Can I explain the four OOP pillars, `super()`, polymorphism, composition and operator overloading?
- ☐ Can I write and explain decorators, decorators with arguments, property/setter and common dunder methods?
- ☐ Can I handle exceptions with `try/except/else/finally` and raise meaningful exceptions?
- ☐ Do I understand `map/filter/reduce`, walrus operator, `*args/**kwargs` and argument unpacking?
- ☐ Can I read/write/append files using `with`, line-by-line processing, `os`, `shutil` and `argparse`?

- ☐ Can I use a third-party package such as requests with timeout + error handling?
- ☐ Can I explain threading, GIL, I/O-bound vs CPU-bound, race conditions, Lock and ThreadPoolExecutor?

Best interview preparation method: For each ★★★ topic: (1) explain it in 30-60 seconds, (2) write a tiny example without notes, (3) state one common trap, and (4) give one real-world use. This converts memorized syntax into interview-ready understanding.

18. Practice Questions by Topic

- Strings: reverse a string; count character frequency; check palindrome; find first non-repeating character.
- Lists: remove duplicates; flatten one level; find second-largest number; split list into chunks.
- Dictionaries: merge dictionaries; group values by key; invert a mapping when values are unique; build a frequency map.
- Sets: find common elements; unique elements; detect duplicates; compare required vs provided permissions.
- Functions: write a retry wrapper; validate arguments; design a function using keyword-only options.
- OOP: design BankAccount, Employee/Manager, Shape hierarchy; implement `__repr__` and `__eq__`; use property validation.
- Decorators: timing decorator; call counter; authorization check; decorator that accepts retry count.
- Exceptions: parse a number safely; define a custom validation exception; use `else/finally` appropriately.
- Files: count ERROR lines from a log without loading the whole file; copy a file; build a CLI with `argparse`.
- Threading: run several simulated I/O tasks with `ThreadPoolExecutor`; protect a shared counter with `Lock`.

Suggested study order: Basics -> operators/conditionals/loops -> strings -> functions -> lists/dicts/sets -> OOP -> exceptions -> decorators/args/kwargs -> files/modules -> threading. Revisit ★★★ topics twice as often as ★ topics.